Wiley
International Journal of Aerospace Engineering
Volume 2024, Article ID 3493676, 14 pages
https://doi.org/10.1155/2024/3493676

WILEY

Research Article

A Fault Diagnosis Method for Avionics Equipment Based on SMOTEWB-LGBM

Gen Li,[1,2] **Wenhai Li**,[1] **Tianzhu Wen**,[1] **Weichao Sun**,[1] **and Xi Tang**[1]

[1]*Aviation Combat Service Academy, Naval Aviation University, Yantai 264001, China*
[2]*Repair Battalion, Unit 77120 of the Chinese People's Liberation Army, Chengdu 611930, China*

Correspondence should be addressed to Gen Li; 504123921@qq.com

Received 12 January 2024; Revised 13 June 2024; Accepted 11 July 2024

Academic Editor: Binbin Yan

To tackle the common issue of imbalanced data classes in the fault diagnosis of avionics equipment, this study proposes a method that integrates the Synthetic Minority Oversampling Technique (SMOTE) with Boosting (SMOTEWB) and the Light Gradient Boosting Machine (LGBM). Initially, SMOTEWB is refined through a "successive one-vs-many balancing strategy," which effectively addresses multiclass sample balance issues. The enhanced data is then employed for pattern recognition and classification using LGBM. Additionally, this study utilizes the Tree-structured Parzen Estimator (TPE) method and five-fold cross-validation to optimize the model's hyperparameters, thus improving diagnostic accuracy. Experimental validation using University of California Irvine (UCI) public datasets and real-world avionics equipment fault data shows that the proposed SMOTEWB-LGBM method outperforms other common methods in handling multiclass imbalanced datasets. This new approach not only enhances fault diagnosis efficiency but also offers a potent solution for similar multiclass imbalance challenges.

Keywords: avionics equipment; class imbalance; ensemble learning; fault diagnosis; multiclass classification; oversampling

1. Introduction

Fault diagnosis is a crucial technology in the Prognostics and Health Management (PHM) of avionics equipment. With the rapid advancement of integrated electronic circuit technology, module-level fault diagnosis has become increasingly significant. The primary aim of fault diagnosis is to isolate faults in the tested object to the corresponding board module. However, the electrical components in avionics equipment typically exhibit tolerances, and, due to the presence of nonlinearities and feedback loops, the fault mechanisms are complex and challenging to accurately capture with traditional mathematical models. These factors collectively heighten the difficulty of fault diagnosis, thereby increasing the demands on diagnostic techniques [1]. In response to these challenges, machine learning technology has emerged as a powerful tool for addressing fault diagnosis issues. At its core, machine learning views fault diagnosis as a pattern recognition problem, constructing diagnostic models through data-driven methods [2].

In the field of avionics equipment fault diagnosis, the issue of imbalanced data classes poses a significant challenge. Avionics equipment typically operates fault-free, resulting in limited and random fault samples, leading to characteristics of class imbalance in the data. Additionally, due to varying failure rates across different modules, there is a significant disparity in the number of samples for each fault class [3]. Traditional classifiers tend to focus on majority class samples, neglecting minority class samples and complicating the accurate recognition of these minority classes [4, 5].

To mitigate this issue, oversampling technology has become a prevalent solution. Oversampling primarily enhances the representation of minority class samples, either through simple repeated sampling or by synthesizing new minority class samples, thus boosting the classifier's ability to distinguish these classes, and enhancing classification performance. Traditional oversampling methods, such as random oversampling, are straightforward but can lead to significant overfitting problems. To address this, Chawla et al. [6] introduced the Synthetic Minority Oversampling Technique (SMOTE) in

2002. SMOTE synthesizes new samples by performing random linear interpolation between minority class samples and their k-nearest neighbors, effectively reducing overfitting issues and pioneering new research avenues in oversampling techniques. Subsequent enhancements to SMOTE, such as Borderline-SMOTE [7], ADASYN [8], and SMOTE with Boosting (SMOTEWB) [9], have further refined the quality and classification performance of samples through various strategies. Notably, SMOTEWB integrates the AdaBoost [10] ensemble learning method, calculates the weights of different classes, and categorizes samples into good, lonely, and bad samples based on their features, employing distinct sampling strategies to effectively minimize noise introduction and enhance the quality of synthesized samples.

This study develops a new fault diagnosis method, SMOTEWB-LGBM (Light Gradient Boosting Machine) for avionics equipment by enhancing the SMOTEWB with a "successive one-vs-many balancing strategy" to effectively manage multiclass imbalanced samples. Following these improvements, the method employs the LGBM ensemble learning classifier to analyze the oversampled data. Additionally, this study utilizes the Tree-structured Parzen Estimator (TPE) algorithm and five-fold cross-validation to optimize the model's hyperparameters, which improves diagnostic accuracy and reduces labor costs. The effectiveness and superiority of this approach have been demonstrated through applications on University of California Irvine (UCI) datasets and actual avionics equipment fault data.

The innovative aspects of this study primarily include the following enhancements: Firstly, the SMOTEWB method is refined with a successive one-versus-many balancing strategy, effectively addressing multiclass imbalances by systematically oversampling each minority class. Secondly, the integration of SMOTEWB's oversampling with LGBM's ensemble learning significantly boosts the accuracy and robustness of fault diagnosis. Lastly, the application of the TPE method for automatic hyperparameter optimization minimizes manual tuning, enhancing the model's overall performance. Collectively, these advancements significantly improve the efficiency and reliability of the fault diagnosis system.

2. A Binary Oversampling Algorithm Based on SMOTEWB

The SMOTE algorithm incorporates k-nearest neighbor and linear interpolation techniques in its oversampling process. It creates new samples at a specific distance between two closely located sample points within the minority sample set to generate an artificial sample that structurally resembles the original. The SMOTE algorithm produces artificial data between existing negative observations by exploiting gaps in the feature space. For a minority class sample x_i, let $\widehat{x}_i$ be a sample of its k-nearest neighbor. Using the Euclidean distance between x_i and $\widehat{x}_i$, along with a random value λ between [0,1], a new synthetic sample x^{syn} is generated, as shown in Equation (1):

$$x^{\text{syn}} = x_i + (\widehat{x}_i - x_i) \times \lambda \tag{1}$$

As indicated in Equation (1), the SMOTE algorithm indiscriminately interpolates all minority class samples, without considering possible noise or outliers, leading to a degree of arbitrariness in sample generation.

SMOTEWB enhances the SMOTE algorithm by integrating AdaBoost to assign weights to various sample types. In each iteration, AdaBoost increases the weight of samples that were misclassified in the previous round. Samples are categorized into three types based on their characteristics, and each type undergoes a distinct sampling strategy to minimize noise introduction and reduce the impact of outliers, thus improving the quality of the synthetic samples. The specific methodology of SMOTEWB is detailed in the literature [9].

The main steps of the SMOTEWB binary oversampling method are as follows:

1. Weight update: Each training sample is assigned a weight that reflects its likelihood of misclassification. The AdaBoost ensemble learning technique is used to increase the weight of samples that were incorrectly classified in the previous round, focusing more on these challenging samples in the subsequent round. The process is outlined below:

For a training set X with corresponding labels Y, assume that there are n samples. Each sample x_i is initially assigned the same weight, as shown in

$$W_i^0 = \frac{1}{n(i = 1, 2, \cdots, n)} \tag{2}$$

To update the weight, the AdaBoost algorithm employs M base classifiers (typically decision trees). For the base classifier $m = 1, 2, \cdots, M$, err_m denotes the sum of the weights of all samples misclassified by m, as shown in

$$err_m = \sum_{i=1}^{n} W_i * I(Y_i \neq F_m(x_i)) \tag{3}$$

Here, the predicted label $F_m(x_i)$ represents the prediction result of the base classifier m, and the true label is Y_i. $I(Y_i \neq F_m(x_i))$ is an indicator function, which is one when $I(Y_i \neq F_m(x_i))$ is true, and zero otherwise.

The sample classification weight W^m is updated as follows:

$$W^m = W^{m-1} * e^{\ln\,(1-err_m/err_m)*I(Y \neq F_m(x))} \tag{4}$$

This formula indicates that if a sample is misclassified, its weight will increase; otherwise, it will remain unchanged.

2. Noise detection: We conduct noise detection on the dataset by establishing a noise threshold for each sample, which depends on the proportion of majority and minority class samples. Samples exceeding this

threshold are classified as noise, while those below it are deemed nonnoise.

The noise threshold $\mathrm{th}_{\min}$ for a minority class sample $x_i^{\min}$ satisfies

$$\mathrm{th}_{\min} = \frac{1}{n}\left(\frac{2 \times n_{\mathrm{maj}}}{n}\right) = \frac{2 \times n_{\mathrm{maj}}}{n^2} \tag{5}$$

In this formula, n_{maj} represents the number of majority class samples, and n is the total number of samples in the training set. For a minority class sample $x_i^{\min}$, if its weight $W_i^{\min}$ exceeds the noise threshold $\mathrm{th}_{\min}$, it is considered a noise sample; otherwise, it is classified as a nonnoise sample. This detection process is critical for categorizing minority class samples in the next step.

For the majority class samples, the noise threshold $\mathrm{th}_{\mathrm{maj}}$ is derived in the following manner:

$$\mathrm{th}_{\mathrm{maj}} = \frac{1}{n}\left(\frac{2 \times n_{\min}}{n}\right) = \frac{2 \times n_{\min}}{n^2} \tag{6}$$

Here, $n_{\min}$ indicates the count of minority class instances, and n denotes the total number of samples in the training set. If the weight W_i^{maj} of a majority class instance x_i^{maj} exceeds $\mathrm{th}_{\mathrm{maj}}$, it is labeled as a noise instance; otherwise, it is a nonnoise instance. This noise detection in the majority class helps reduce adverse effects on the oversampling process for minority class instances, thereby improving the quality of the synthetic samples generated.

3. Classification of samples: Using the results from noise detection, we dynamically determine the appropriate number of neighboring samples, denoted as k. Minority class samples are then categorized into three groups: good samples, lonely samples, and bad samples:

Good samples are nonnoise samples that include at least one majority class sample within their k-nearest neighbors.

Lonely samples are nonnoise samples that lack majority class samples within their k-nearest neighbors.

Bad samples are those identified as noise samples.

4. Sample generation: To address the imbalance between minority and majority class samples, we generate new minority class samples based on a predetermined ratio.

Good samples have additional samples randomly generated using the SMOTE algorithm, which synthesizes new samples between the original sample and a majority class sample from its k-nearest neighbors.

Lonely samples are simply replicated to create new samples.

By implementing these procedures, SMOTEWB effectively tackles the issue of imbalanced binary data through strategic oversampling and the creation of synthetic samples from the minority class, thus achieving a balanced distribution of binary data.

3. Framework for Multiclass Fault Diagnosis Using the SMOTEWB-LGBM Model

3.1. Oversampling of Multiclass Samples Using the SMOTEWB. Although SMOTEWB is an effective technique for binary oversampling, it is not ideally suited for addressing imbalanced datasets with multiple classes, such as those frequently encountered in avionics equipment fault diagnosis. Specifically, datasets in this field often exhibit a significant disparity between the number of samples in the normal operating class and those in various fault classes. This variation arises due to the differing frequencies and detectability of faults across the diverse avionics modules. To tackle this challenge, our study proposes a successive one-versus-many balancing strategy that utilizes SMOTEWB to oversample the minority classes, excluding the class with the highest number of samples. This strategy aims to equalize the sample sizes across all classes, thereby enhancing the accuracy and reliability of the multiclass imbalanced dataset. The implementation process is illustrated in Figure 1.

The enhanced SMOTEWB employs a successive one-versus-many balancing strategy to systematically oversample each minority class in turn. After oversampling a minority class, the new synthetic samples are added to the dataset. The steps involved are as follows:

1. Rearrange the initial dataset: Organize the dataset based on the number of samples within each class and assign appropriate labels. In a dataset comprising p classes of samples, denoted as $1, 2, \cdots, p$, the samples within each class are reordered in descending order by sample count, resulting in a newly arranged set of samples denoted as $1', 2', \cdots, p'$. Notably, class $1'$ has the highest number of samples, while class p' has the fewest.

2. Calculate the number of synthetic samples: Starting with the $2'$-th class, generate synthetic samples in sequence until the number of samples in this class matches that of the largest class $n_{1'}$, which corresponds to the $1'$-th class. The required number of synthetic samples $n_{i'}^{\mathrm{gen}}$ for the current class i is

$$n_{i'}^{\mathrm{gen}} = n_{1'} - n_{i'} \tag{7}$$

where $n_{i'}$ represents the initial number of samples in the minority class i' being processed.

3. Perform successive SMOTEWB oversampling: Use the $2'$-th class as the minority class and the $1'$-th class as the majority class. Generate $n_{2'}^{\mathrm{gen}}$ synthetic samples for the minority class using the SMOTEWB algorithm and add them to the $2'$-th class. For the $3'$-th class and onwards, use the i-th class as the minority class and the set of $1', \cdots, i' - 1$ original and synthetic samples as the majority class. Generate $n_{i'}^{\mathrm{gen}}$ synthetic samples for the i'-th class using the SMOTEWB

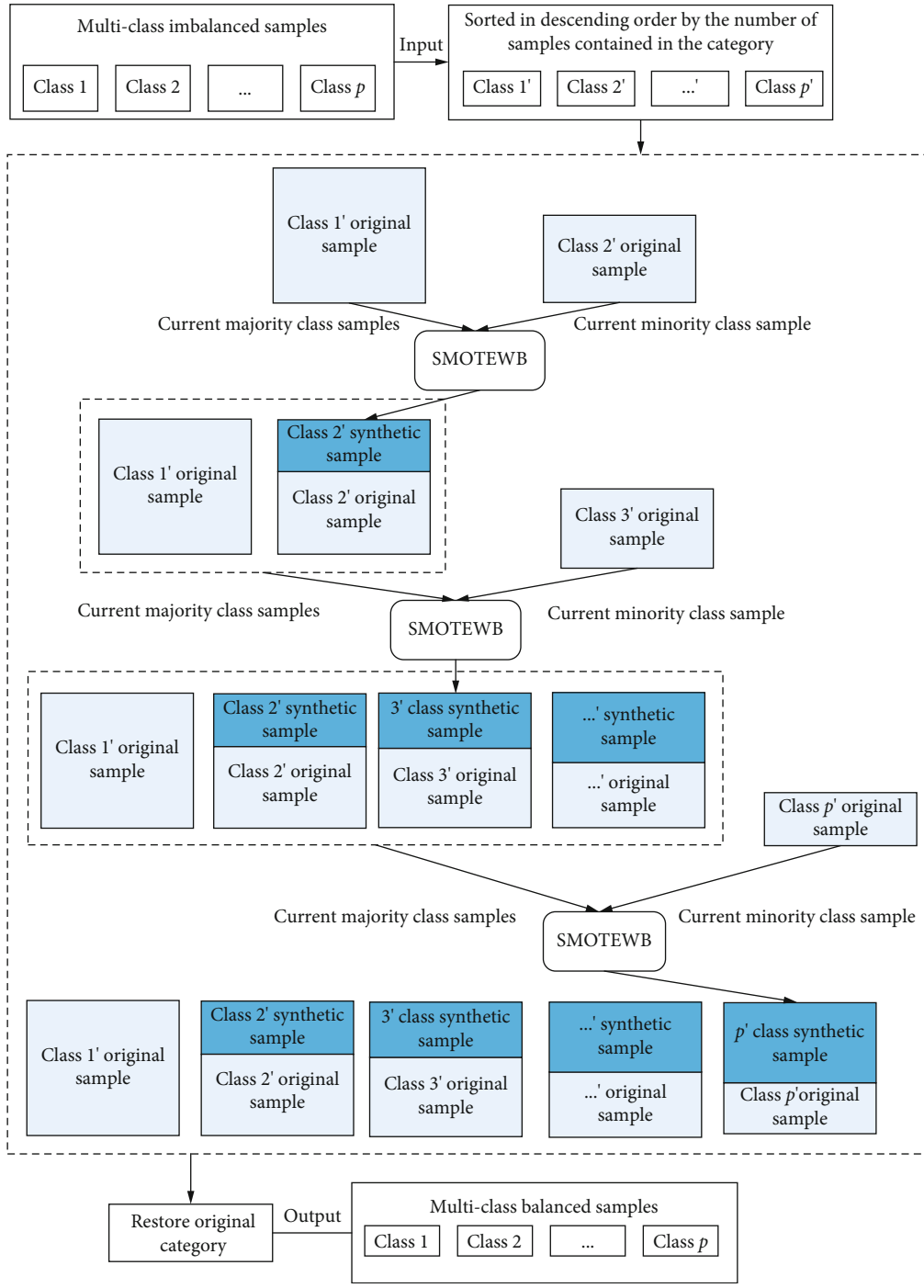

Figure 1: SMOTEWB multiclass oversampling using successive one-versus-many balancing strategy.

algorithm and add them to the i'-th class. Stop after generating $n_{i'}^{\text{gen}}$ samples for the p'-th class. The final number of samples in each class will be equal to the number of samples in the $1'$-th class.

4. Combine all class samples: Both original and synthetic samples are combined according to their original class labels. In the final oversampled dataset, each sample retains the same class label as in the original data.

The successive one-versus-many strategy is designed to diminish the disparity in sample numbers across classes by progressively increasing the samples in each minority class. This approach considers the oversampling outcomes of previous classes, which better preserves the characteristics of the data distribution. Thanks to the robust noise handling capabilities of SMOTEWB, when combined with this strategy, it can generate higher-quality synthetic samples for minority classes, thereby achieving a more balanced distribution for multiclass imbalanced data.

By using the successive one-versus-many strategy to enhance SMOTEWB's multiclass oversampling, we can achieve a dataset where each class has equal numbers of samples. This not only increases the quantity of training samples but also ensures a balance across different types of data, providing higher-quality training samples for subsequent LGBM pattern recognition.

3.2. LGBM Method for Fault Pattern Recognition. LGBM is a distributed system developed by Ke et al. [11] from Microsoft Research Asia in 2017. It is an enhancement of Gradient Boosting Decision Tree (GBDT) [12], known for its excellent classification capabilities [13]. After oversampling multiclass samples using the SMOTEWB model, the balanced data is fed into the LGBM model for training, enabling precise fault pattern recognition.

LGBM operates on the principles of the boosting framework additive model and the forward propagation algorithm. Consider the training dataset to be denoted as $D = \{(x_1, y_1), (x_2, y_2), \cdots, (x_n, y_n)\}, x_i \in X, y_i \in Y$, with X representing the sample feature inputs, and Y, the corresponding labels. The loss function is represented by $L(y, F(x))$, where $F(x)$ is the final, strong model obtained by linearly combining all models through weights. $F(x)$ is expressed as

$$F(x) = \sum_{m=1}^{M} \beta_m f(x; \gamma_m) \tag{8}$$

Here, $f(x; \gamma_m)$ represents the weak model, β_m represents the weight coefficient, and γ_m represents the parameters of the weak model.

During the iteration process, if the model at the $(m\text{-}1)$-th iteration is $f_{m-1}(x)$ and the loss function is $L(y_{m-1}, f_{m-1}(x))$, the goal for the m-th iteration is to identify a weak learner $h_m(x)$ that minimizes the loss function $L(y_m, f_m(x)) = L(y_m, f_{m-1}(x) + h_m(x))$ in the current iteration. Essentially, in the m-th iteration, the objective is to discover a decision tree model $h_m(x)$ that significantly reduces the sample loss. To effectively lower model loss and adapt to various tasks, different loss functions are often employed. The negative gradient of the loss function at the current function $f(x) = f_{m-1}(x)$ is typically used to approximate the residual. This is expressed as

$$r_{mi} = -\left[\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)}\right]\Big|_{f(x)) f_{m-1}(x)} \tag{9}$$

Based on the data (x_i, r_{mi}), the m-th decision tree can be fitted, and its corresponding leaf node area is R_{mj}, where $j = 1, 2, \cdots, J$ represents the number of leaf nodes. For each sample in the leaf node, linear search is applied to determine the output value c_{mj} that minimizes the loss function, expressed as

$$c_{mj} = \arg \min_{c} \sum_{x_i \in R_{mj}} L(y, f_{m-1}(x_i) + c) \tag{10}$$

Here, c is a constant value that can minimize the loss function.

At this stage, the decision tree model for the m-th round is

$$h_m(x) = \sum_{j=1}^{J} c_{mj} I(x_i \in R_{mj}) \tag{11}$$

Here, $I(x_i \in R_{mj})$ is an indicator function. If x_i is in the R_{mj} region, it results in one; otherwise, it is zero.

The model is then updated to

$$f_m(x) = f_{m-1}(x) + \sum_{j=1}^{J} c_{mj} I(x_i \in R_{mj}) \tag{12}$$

Finally, the initial decision tree and the decision trees from each iteration are aggregated to produce the learner:

$$F(x) = f_0(x) + \sum_{m=1}^{M} \sum_{j=1}^{J} c_{mj} I(x_i \in R_{mj}) \tag{13}$$

During the fitting process of decision trees, LGBM converts continuous floating-point eigenvalues into q integers, constructs a histogram of width q, and identifies the optimal segmentation point, substantially reducing storage requirements. The leaf-wise growth strategy, utilized in node splitting, minimizes search and splitting time while increasing accuracy by focusing on the leaves with the highest splitting gain. Furthermore, techniques such as gradient-based one-sided sampling and exclusive feature bundling in LGBM maximize data utilization and accelerate training efficiency.

3.3. Stratified K-Fold Cross-Validation. K-fold cross-validation divides the dataset into K subsets of equal size, selecting one subset to assess the model's performance, while employing the remaining K-1 subsets for training. Given the imbalanced nature of the dataset, the stratified K-fold method is utilized to ensure that the proportion of each class in each subset mirrors that in the original dataset [14]. This approach effectively utilizes the data and mitigates the risk of overfitting. In this study, K is set to five, meaning that five-fold cross-validation is implemented. The training and validation sets are randomly divided into five parts, where each of the five subsets is oversampled using SMOTEWB to create five oversampled subsets.

The schematic diagram of five-fold cross-validation is shown in Figure 2, and to validate the effectiveness of this method, each 50% unsampled subset is used in turn as a validation set, with the corresponding four oversampled subsets serving as the training set. After completing five validation rounds, the average result of these validations represents the outcome of the cross-validation.

3.4. TPE Hyperparameter Optimization. In 2011, Bergstra et al. [15] from Harvard University introduced the TPE, a method that models the relationship between hyperparameters using a tree structure and effectively addresses

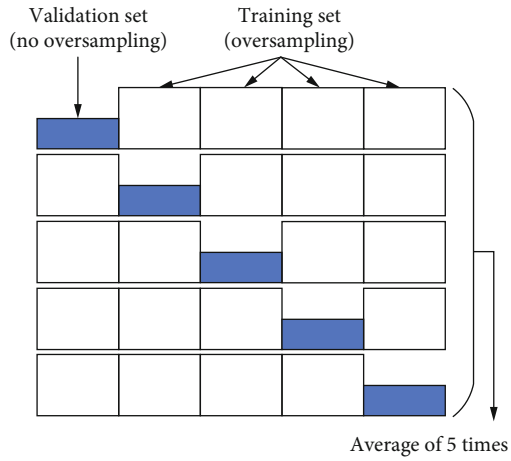

Figure 2: Schematic diagram of five-fold cross-validation.

multidimensional optimization challenges. With only a few iterations, TPE can achieve superior hyperparameter selection. Consequently, TPE is an optimal method for tuning the hyperparameters of the SMOTEWB-LGBM model, requiring minimal computing resources while ensuring efficient operation. This method is adaptable to different objective functions, further enhancing the quality of hyperparameter selection.

The TPE algorithm defines $p(x|y)$ using two density functions as follows:

$$p(x|y) = \begin{cases} l(x) & \text{if} \quad y < y^* \\ g(x) & \text{if} \quad y \geq y^* \end{cases} \tag{14}$$

Here, x represents the observation point, which is the hyperparameter vector of the model to be optimized; y is the observation value, the outcome of the objective function (loss or evaluation function) for the given parameter x; and y^* is a threshold, representing a specific quantile of the TPE algorithm used to divide the observations into two density functions, $l(x)$ and $g(x)$, with the sum ranging from zero to one. The algorithm sets y^* based on existing observations, segregating them into two distinct density functions: $l(x)$ is formed by those observations $\{x_{(i)}\}$ whose loss is $f(x_{(i)}) < y^*$, and $g(x)$ consists of the remaining observations. TPE employs Bayesian optimization principles to minimize ineffective search space.

The Expected Improvement (EI) strategy employed by the TPE algorithm generates new observation points aimed at maximizing EI. The Bayesian approach optimizes the EI acquisition function, letting $p(y)p(x|y)$ be $p(x, y)$, thus defining EI as follows:

$$\mathrm{EI}_{y^*}(x) = \int_{-\infty}^{y^*} (y^* - y)p(y|x)dy = \int_{-\infty}^{y^*} (y^* - y)\frac{p(x|y)p(y)}{p(x)}dy \tag{15}$$

Let $\gamma = p(y < y^*)$, and to simplify the above formula, construct the denominator as $p(x) = \int p(x|y)p(y)dy = \gamma l(x) + (1-\gamma)g(x)$.

Secondly, for the molecule, we can get the formula

$$\begin{aligned} \int_{-\infty}^{y^*} (y^* - y)p(x|y)p(y)dy &= l(x)\int_{-\infty}^{y^*} (y^* - y)p(y)dy \\ &= \gamma y^* l(x) - l(x)\int_{-\infty}^{y^*} p(y)dy \end{aligned} \tag{16}$$

Consequently, EI can be simplified to

$$\mathrm{EI}_{y^*}(x) = \frac{\gamma y^* l(x) - l(x)\int_{-\infty}^{y^*} p(y)dy}{\gamma l(x) + (1-\gamma)g(x)} \propto \left(\gamma + \frac{g(x)}{l(x)}(1-\gamma)\right)^{-1} \tag{17}$$

From this formula, maximizing EI involves ensuring that the probability of $l(x)$ is high and that of $g(x)$ is low. In each iteration, the algorithm returns the candidate hyperparameter x^* with the highest EI value, $x^* = \arg\max \mathrm{EI}_{y^*}$. During the maximization process, the hyperparameter x that achieves the highest $l(x)$ and the lowest $g(x)$ probabilities obtains the highest EI value. The TPE algorithm uses $l(x)$ and $g(x)$ to generate a collection of hyperparameter samples and evaluates x by the ratio of $l(x)/g(x)$. Each iteration returns the point with the highest EI value. Using this approach, we identify the optimal hyperparameters for the SMOTEWB-LGBM model. In optimizing these parameters, the new x is employed to construct the fault diagnosis model for the SMOTEWB-LGBM algorithm, and the observation y is obtained through training convergence. The new observation point is then compared with the original, updating the probabilistic surrogate model to select the hyperparameter x corresponding to the best result.

In this study, we utilize the microaverage F1 (Micro-F1) score from the five-fold cross-validation as the objective function for optimizing parameters through the TPE method. This method helps identify the optimal hyperparameters by maximizing the aforementioned score.

We apply the TPE algorithm to optimize hyperparameters for both SMOTEWB and LGBM simultaneously. Once optimized, the LGBM hyperparameters are then used in the LGBM model for classification diagnosis testing. Although the SMOTEWB hyperparameters do not participate directly in the testing phase, they play a crucial role during the training phase by altering the distribution of the training data through the generation of synthetic samples. This alteration aids in training a more robust LGBM classification model. The framework of this SMOTEWB-LGBM multiclass fault diagnosis is illustrated in Figure 3.

4. Evaluation Indicators and Diagnostic Process

4.1. Evaluation Indicators for Class Imbalance Problem. In traditional classification problems, classification accuracy is commonly used as a key metric to evaluate classifier performance. However, when dealing with imbalanced datasets, relying solely on classification accuracy may not accurately reflect the overall classifier performance. To overcome this

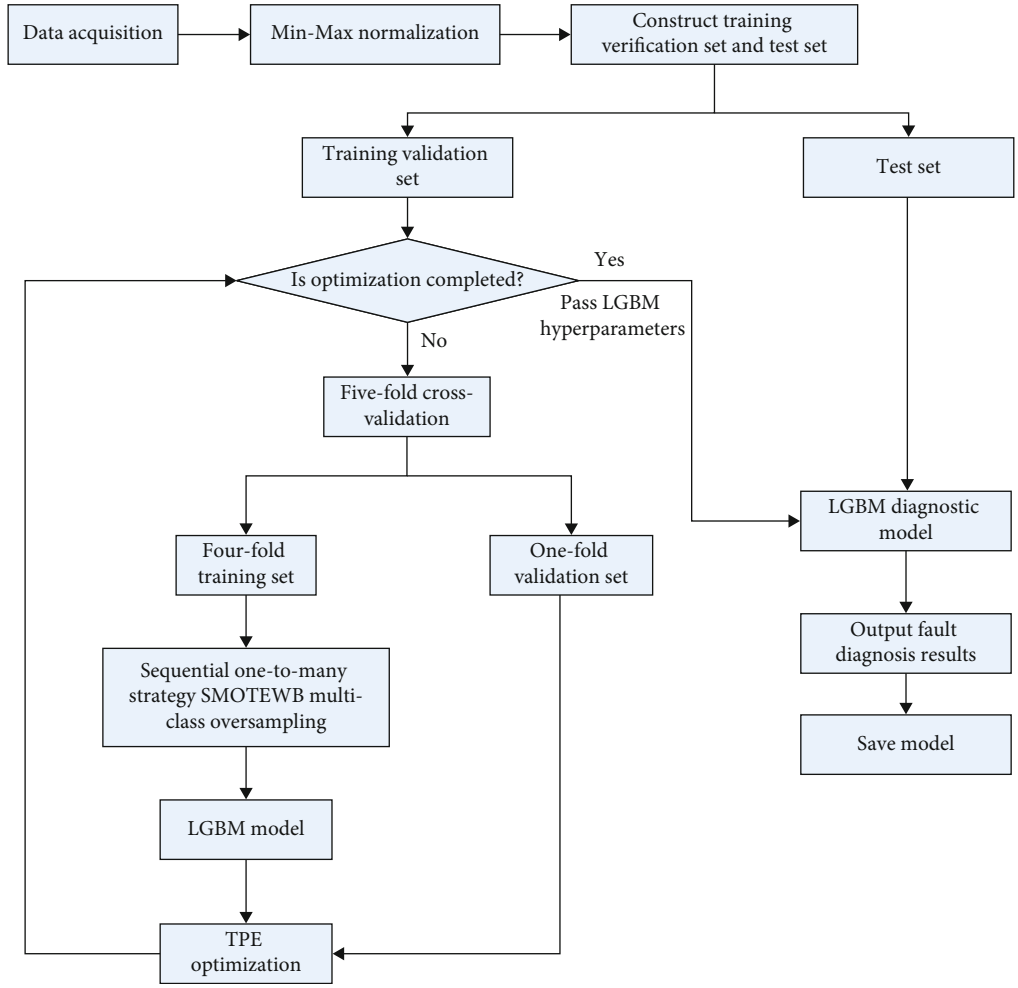

Figure 3: SMOTEWB-LGBM multiclass fault diagnosis framework.

limitation, this study incorporates the weighted precision (Weighted-P) as suggested in the literature [16], along with the macroaverage F1 (Macro-F1) score and the Micro-F1 score from the literature [17] as metrics to evaluate classification performance. The confusion matrix for binary classification is presented in Table 1.

Precision is the ratio of examples where the predicted value and the true value are both positive, denoted by a symbol ω. Recall represents the ratio of examples where the true value is positive that are correctly predicted as positive, denoted by a symbol σ. The F1 score in class classification is defined as the harmonic mean of precision and recall. The relevant definitions are as follows:

$$\omega = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP}} \tag{18}$$

$$\sigma = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FN}} \tag{19}$$

$$\mathrm{F1} = \frac{2\omega\sigma}{\omega + \sigma} \tag{20}$$

Table 1: Confusion matrix for binary classification results.

Real situation	Classification results	
	Normal	Abnormal
Normal	TP (true positive)	FN (false negative)
Abnormal	FP (false positive)	TN (true negative)

For p-class multiclass classification problems, the weighted average precision (Weighted-P) is given as

$$\text{Weighted-P} = \sum_{i=1}^{m} \omega_i \cdot \frac{n_i}{n} \tag{21}$$

where n_i denotes the number of samples in class i and n is the total number of samples.

To extend the F1 score to multiclass classification, two types of averaging are usually used, namely, Macro-F1 score and Micro-F1 score.

Table 2: Relevant information of UCI datasets.

Dataset name	Number of samples	Number of classes	Number of features	Number of samples in each class
Balance-scale	625	3	4	288, 49, 288
Diabetes	768	2	8	500, 268
Dermatology	529	6	34	112, 61, 72, 49, 52, 20
Cell-cycle	217	4	17	47, 31, 18, 121
User-knowledge	403	5	5	102, 129, 122, 24, 26
Glass	213	6	9	69, 76, 17, 13, 9, 29

Table 3: Experimental results of UCI datasets.

Datasets	Models	Weighted-P	Macro-F1	Micro-F1
Balance-scale	SMOTEBoost	0.798 3	0.597 9	0.840 0
	RUSBoost	0.853 4	0.695 1	0.808 0
	Adacost	0.780 4	0.588 8	0.848 0
	ImECOC	0.820 4	0.615 9	**0.880 0**
	LGBM	0.831 1	0.651 0	0.864 0
	SMOTEWB-LGBM	**0.863 4**	**0.696 3**	0.832 0
Diabetes	SMOTEBoost	0.795 2	0.765 7	0.7857
	RUSBoost	0.701 7	0.654 3	0.675 3
	Adacost	0.781 5	0.724 0	0.733 8
	ImECOC	0.794 9	0.766 7	0.798 7
	LGBM	0.797 8	0.771 7	0.798 7
	SMOTEWB-LGBM	**0.809 8**	**0.784 2**	**0.805 2**
Dermatology	SMOTEBoost	0.928 3	0.935 4	0.918 9
	RUSBoost	0.922 0	0.892 4	0.890 4
	Adacost	0.948 0	0.932 3	0.945 2
	ImECOC	0.959 2	0.956 3	0.959 4
	LGBM	0.945 5	0.937 6	0.945 2
	SMOTEWB-LGBM	**0.975 6**	**0.969 2**	**0.973 0**
Cell-cycle	SMOTEBoost	0.550 2	0.405 6	0.454 5
	RUSBoost	0.643 3	0.500 7	0.659 1
	Adacost	0.687 0	0.514 9	0.636 4
	ImECOC	0.637 9	0.470 2	0.659 1
	LGBM	0.502 9	**0.618 2**	0.636 4
	SMOTEWB-LGBM	**0.834 8**	0.593 9	**0.727 0**
User-knowledge	SMOTEBoost	0.912 9	0.847 3	0.913 6
	RUSBoost	0.695 6	0.497 8	0.691 4
	Adacost	0.654 1	0.394 7	0.567 9
	ImECOC	0.823 2	0.672 3	0.864 2
	LGBM	0.917 1	0.799 1	0.925 9
	SMOTEWB-LGBM	**0.952 3**	**0.910 7**	**0.950 6**
Glass	SMOTEBoost	0.790 6	0.753 2	0.744 2
	RUSBoost	0.513 0	0.302 6	0.465 1
	Adacost	0.765 9	0.746 1	**0.790 8**
	ImECOC	0.715 6	0.623 8	0.744 2
	LGBM	0.806 3	0.663 5	0.790 7
	SMOTEWB-LGBM	**0.813 5**	**0.756 2**	0.767 4

Numbers in bold represent the best results obtained.

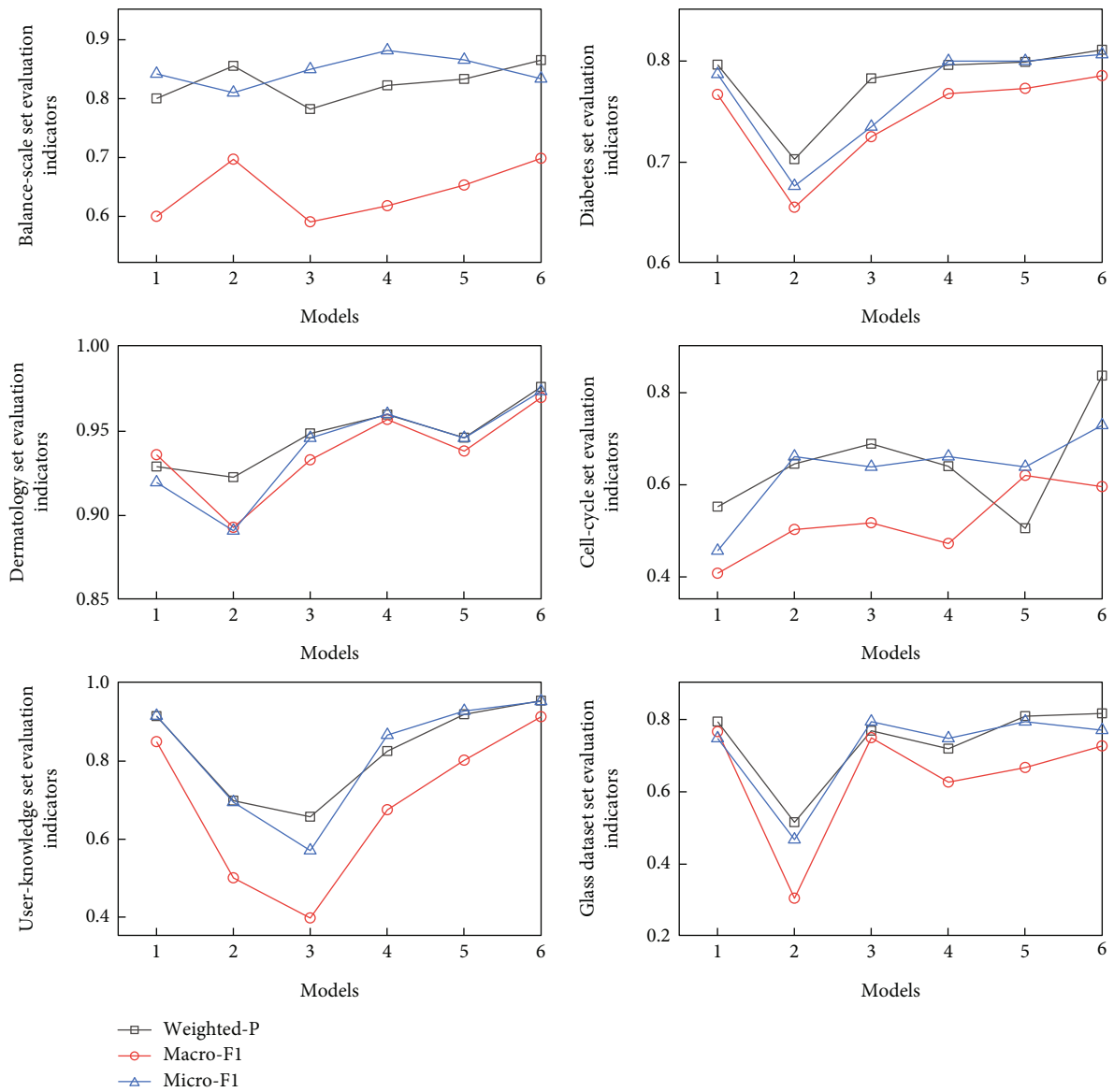

Figure 4: Graphical representation of UCI experimental results. The models represented by the numbers are: 1:SMOTEBoost; 2:RUSBoost; 3:Adacost; 4:ImECOC; 5:LGBM; 6:SMOTEWB-LGBM.

Macro-F1 satisfies

$$F_i = \frac{2\omega_i \sigma_i}{\omega_i + \sigma_i} \tag{22}$$

$$\text{Macro-F1} = \frac{1}{p}\sum_{i=1}^{p} F_i \tag{23}$$

Micro-F1 is formulated into the following functions:

$$\omega = \frac{\sum_{i=1}^{p} \text{TP}_i}{\sum_{i=1}^{p} (\text{TP}_i + \text{FP}_i)} \tag{24}$$

$$\sigma = \frac{\sum_{i=1}^{p} \text{TP}_i}{\sum_{i=1}^{p} (\text{TP}_i + \text{FN}_i)} \tag{25}$$

$$\text{Micro-F1} = \frac{2\omega\sigma}{\omega + \sigma} \tag{26}$$

These three indicators offer different perspectives to evaluate model performance. The weighted average precision and Micro-F1 score are more sensitive to large classes, while the Macro-F1 score treats all classes as equally important. The three indicators are employed to facilitate a comprehensive analysis of algorithm performance.

4.2. Fault Diagnosis Process. The fault diagnosis process of SMOTEWB-LGBM is as follows.

Step 1. Data preprocessing including Min–Max normalization.

Step 2. Application of the SMOTEWB method with a one-to-many balancing strategy to oversample multiclass imbalanced data, equalizing the sample count across all classes.

Step 3. Pattern recognition using the LGBM ensemble learning method on the balanced data.

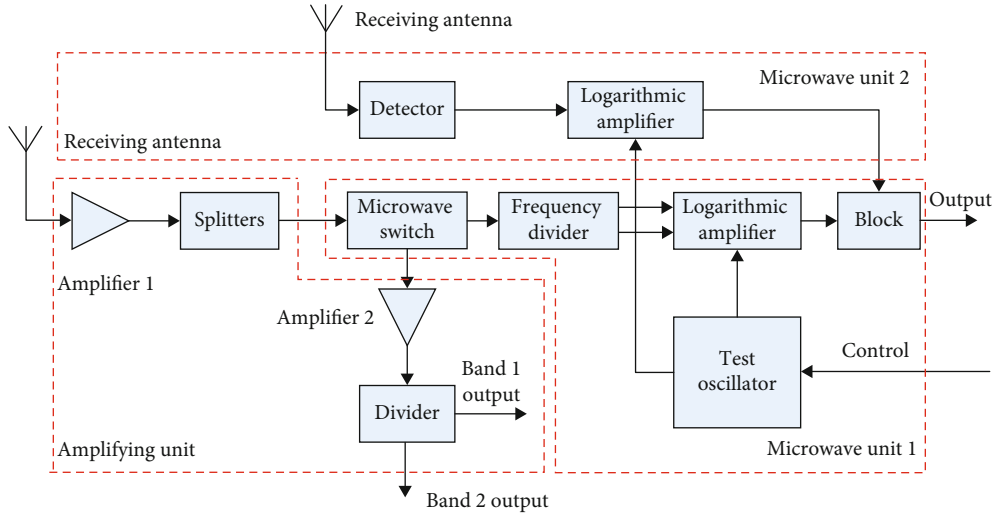

Figure 5: Schematic diagram of the front-end receiver.

Table 4: Model hyperparameters obtained by TPE optimization.

Category of parameters	TPE tuning parameters	Range of parameter values	Type of parameter values	Optimal hyperparameter values
SMOTEWB parameters	n_iters	(30, 100)	int	96
	max_depth	(5, 30)	int	16
LGBM parameters	lambda_l1	(0, 10)	Float	0.1049
	lambda_l2	(0, 10)	float	0.2042
	num_leaves	(3, 128)	int	17
	feature_fraction	(0.5, 1)	Float	0.5204
	bagging_fraction	(0.5, 1)	Float	0.9395
	learning_rate	(0, 1)	Float	0.0979

Step 4. Optimization of SMOTEWB and LGBM hyperparameters using the TPE algorithm and five-fold cross-validation, followed by model output.

Step 5. Determination of the mode category for new samples and identification of the fault mode.

5. Experiment and Discussion

The SMOTEWB-LGBM method proposed in this study was initially tested on datasets from the UCI and subsequently on avionics equipment for diagnostic experiments. It was also compared with other multiclassification algorithms designed for imbalanced data, such as SMOTEBoost [18], RUSBoost [19], Adacost [20], ImECOC [21], as well as LGBM, which is an ensemble learning method suitable for such data. The TPE algorithm was implemented using the Optuna [22] hyperparameter tuning framework.

The experiments were conducted in the following environment: an Intel Core i9-13900HX processor with a base frequency of 2.20 GHz and a max turbo frequency of 5.60 GHz, and 32 GB of memory. The software environment included a Windows 11 operating system and the PyCharm 2023.1 integrated development environment, using Python version 3.8.5.

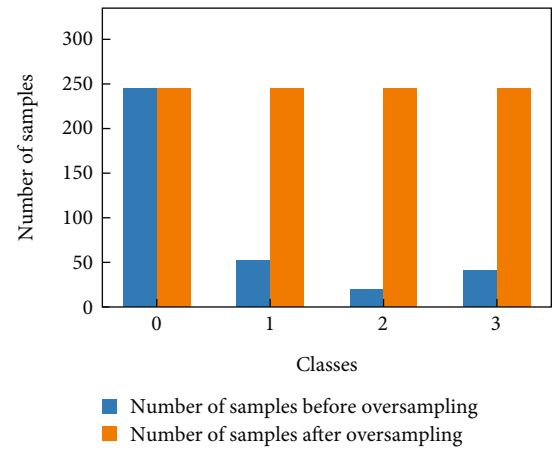

Figure 6: Comparison of the class number before and after oversampling.

5.1. Experiments on UCI Datasets. The UCI datasets, which are publicly available and commonly used to benchmark algorithm performance in machine learning, were selected to test the effectiveness of the proposed method. Six datasets

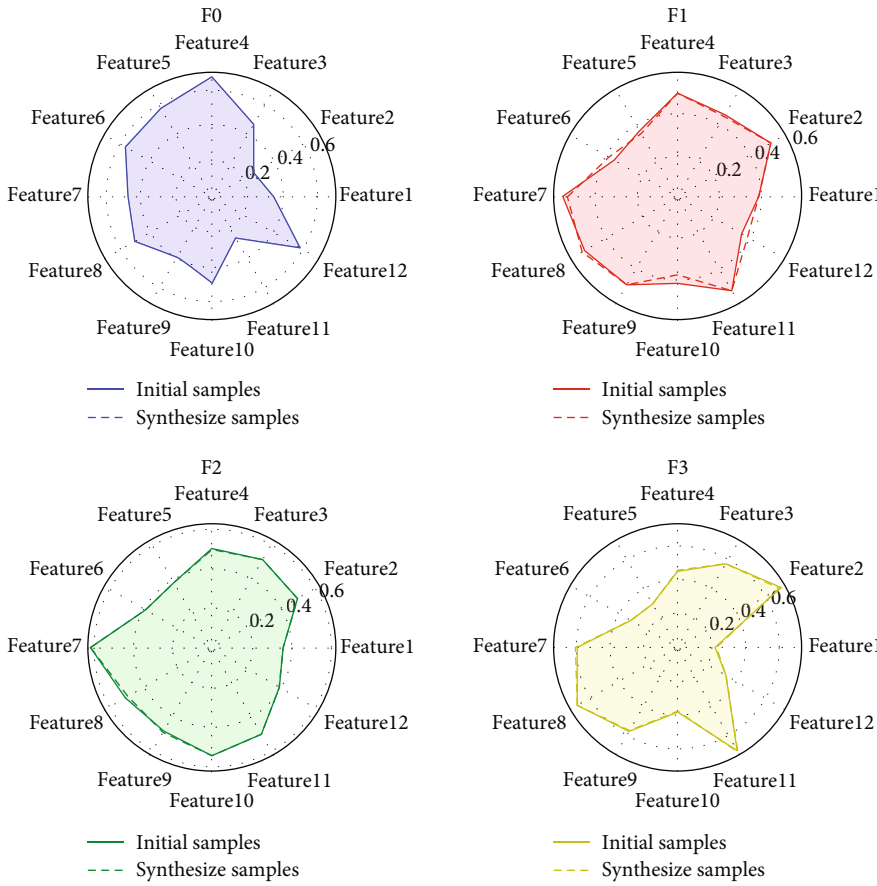

Figure 7: Mean values of F0–F3 class sample features before and after SMOTEWB oversampling.

with varying imbalances were used: Balance-scale, Diabetes, Dermatology, Cell-cycle, User-knowledge, and Glass. The details of these datasets are provided in Table 2.

Eighty percent of the data from each dataset were randomly selected as training samples, with the remaining 20% serving as the test set. To minimize the impact of randomness, 10 experiments were conducted on each dataset, and the results were averaged. The findings from these experiments are displayed in Table 3, with the optimal values for each indicator highlighted in bold. Corresponding graphical representations are shown in Figure 4.

As depicted in Table 3 and Figure 4, the SMOTEWB-LGBM method achieved the highest scores across all evaluation metrics for three datasets—namely, Diabetes, Dermatology, and User-knowledge, and registered the best results in two metrics across three datasets, specifically Balance-scale, Cell-cycle, and Glass. Notably, in the Dermatology dataset, it significantly outperformed other methods. This superior performance underscores the effectiveness of combining the SMOTEWB oversampling method, which adeptly minimizes noise introduction, with the robust classification capabilities of the LGBM ensemble learning method, thus significantly enhancing the classification outcomes for multiclass imbalanced data. The success of this method across multiple datasets also demonstrates its broad applicability.

5.2. Front-End Receiver Fault Diagnosis Experiment. To assess the practical effectiveness of the SMOTEWB-LGBM diagnostic model in avionics equipment diagnosis, we applied it to the fault diagnosis of the front-end receiver in a specific type of aircraft electronic countermeasure system. Figure 5 illustrates the basic working principle of the front-end receiver. The primary function of the front-end receiver is to amplify and filter signals within its operational frequency range, convert them into video signals containing threat information, and forward these signals to both the signal processor for further processing and to the central receiver along with the low-power radio frequency unit. These radio frequency signals carry essential frequency band information pertaining to threat radars.

This study conducted module-level fault diagnosis on the front-end receiver. The diagnosis is distinguished between a normal mode and three fault modes based on the receiver's functionalities. The modes are denoted as follows: normal mode (F0) and three fault modes—amplification unit module failure (F1), microwave unit 1 module failure (F2), and microwave unit 2 module failure (F3). The diagnostic parameters included 12 test measurements, representing 12-dimensional sample features: sensitivity at five frequency points, dynamic range at five frequency points, and gain at two radio frequencies.

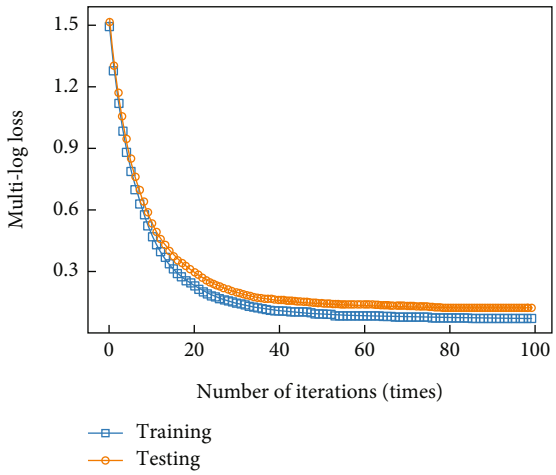

Figure 8: Loss curve of the model.

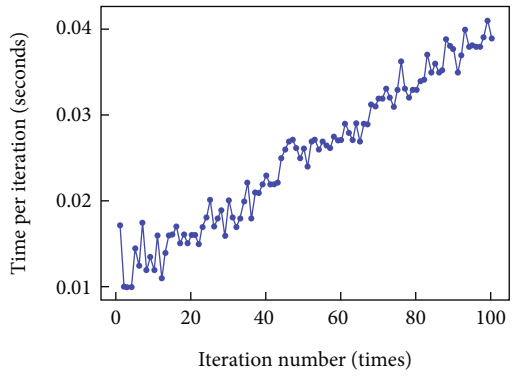

Figure 9: Time spent on each iteration of the model training process.

Data collection utilized the Automatic Test System (ATS) for this type of aviation equipment, employing standard signal sources, power meters, and spectrum analyzers as specified in the factory maintenance manual. A total of 450 datasets were compiled, with 306 sets of normal samples (F0) and 63, 31, and 50 sets of fault samples for F1, F2, and F3 modes, respectively. This dataset exhibited a pronounced class imbalance.

The data were randomly divided into 80% training samples and 20% test samples. Using the Optuna hyperparameter tuning framework, the TPE algorithm, and stratified five-fold cross-validation, we simultaneously tuned the parameters of the SMOTEWB oversampling algorithm and the LGBM pattern recognition method. The four-fold training set comprised the sample post-SMOTEWB oversampling, while the validation set utilized the original data samples. SMOTEWB required tuning for two parameters, and LGBM required tuning for six parameters. Table 4 displays the optimal hyperparameters obtained through this optimization.

Figure 6 shows the distribution of class samples before and after SMOTEWB oversampling. Initially, there was a marked imbalance among the classes, with the F0 class having the most samples and the F2 class the fewest—over three times fewer than F0. This imbalance hindered optimal pattern

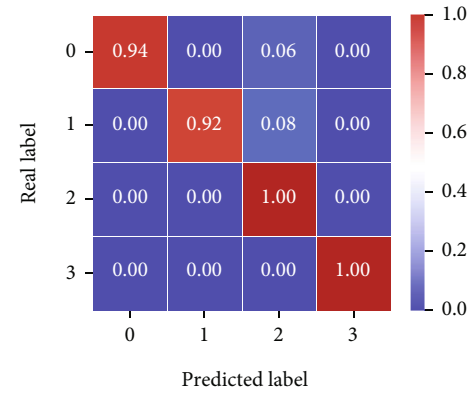

Figure 10: Model confusion matrix diagram.

Table 5: Front-end receiver experimental results.

Models	Weighted-P	Macro-F1	Micro-F1
SMOTEBoost	0.923 7	0.892 8	0.918 4
RUSBoost	0.834 6	0.730 1	0.836 7
Adacost	0.857 1	0.794 8	0.816 3
ImECOC	0.865 8	0.781 0	0.836 7
LGBM	0.857 1	0.786 8	0.857 1
SMOTEWB-LGBM	**0.960 5**	**0.951 0**	**0.959 2**

Numbers in bold represent the best results obtained.

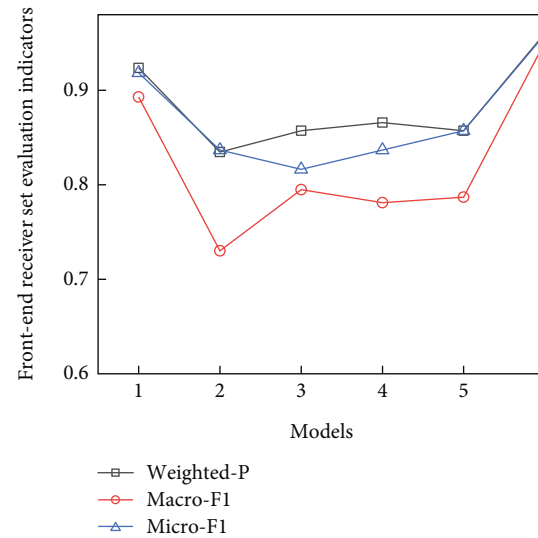

Figure 11: Experimental results of front-end receiver. The models represented by the numbers are: 1:SMOTEBoost; 2:RUSBoost; 3:Adacost; 4:ImECOC; 5:LGBM; 6:SMOTEWB-LGBM.

recognition. However, after employing SMOTEWB's successive one-versus-many strategy for oversampling, the number of samples in each class matched that of the largest class before oversampling, significantly increasing the size of the training dataset and improving the LGBM's classification performance.

Figure 7 displays a radar chart comparing the mean values of different features for the original samples and the new samples generated by SMOTEWB oversampling. It is

Table 6: Experimental results of front-end receiver across different data split ratios.

Algorithm	Weighted-P			Macro-F1			Micro-F1		
Data segmentation ratio	8 : 2	7 : 3	6 : 4	8 : 2	7 : 3	6 : 4	8 : 2	7 : 3	6 : 4
SMOTEBoost	0.923 7	0.933 5	0.920 6	0.892 8	0.848 5	0.8317	0.918 4	0.933 3	0.922 2
RUSBoost	0.834 6	0.881 0	0.822 3	0.730 1	0.695 8	0.616 2	0.836 7	0.822 2	0.838 9
Adacost	0.857 1	0.877 4	0.856 2	0.794 8	0.670 0	0.6480	0.816 3	0.800 0	0.783 3
ImECOC	0.865 8	0.820 7	0.819 2	0.781 0	0.569 8	0.5019	0.836 7	0.807 4	0.788 9
LGBM	0.857 1	0.937 1	0.928 2	0.786 8	0.846 2	**0.838 6**	0.857 1	0.933 3	0.922 2
SMOTEWB-LGBM	**0.960 5**	**0.946 9**	**0.928 7**	**0.951 0**	**0.870 4**	0.801 4	**0.959 2**	**0.948 1**	**0.927 8**

Numbers in bold represent the best results obtained.

evident that no new samples were generated for the F0 class, which was the largest class. In contrast, new samples were created for the F1, F2, and F3 classes, with the mean values of the features for these new samples closely mirroring those of the original samples. This similarity indicates that the new samples preserved the characteristics of the original data and maintained a similar distribution, thereby more accurately representing the original minority samples.

Figure 8 illustrates the trend of the multiclass logarithmic loss value from the first to the 100th iteration of the SMOTEWB-LGBM model under the optimal hyperparameter combination. The chart shows a rapid decrease in the training set loss, stabilizing at approximately 0.08284 after about 60 iterations. The test set loss converged to 0.12148. These results indicate that the default 100 iterations were adequate. The slightly higher loss on the test set compared to the training set, while still low, demonstrates that the model performed robustly on unseen data without overfitting. The minor difference between the loss values of the training and test sets confirms that the model was well-fitted, avoiding overfitting, and was effective in classifying and identifying faults.

Figure 9 shows the time spent on each iteration of the model training process. The time per iteration follows a nonlinear increasing trend, suggesting that as the number of iterations grows, so does the complexity and computational load of the training process. This nonlinear increase may stem from the dynamic allocation of computational resources. Despite this rise, the overall time per iteration remains within an acceptable range, from approximately 0.01–0.041 s per iteration. The brief testing time of the model indicates its suitability for fault diagnosis in avionics equipment, fulfilling engineering requirements.

To better assess the fault diagnosis performance of the classification model under the imbalanced dataset, Figure 10 presents the model's classification confusion matrix. The y-axis in the figure represents the actual fault labels of the data, while the x-axis shows the predicted labels.

The model achieved 100% prediction accuracy for the F2 and F3 classes, and 94% and 92% classification accuracy for the F0 and F1 classes, respectively. The confusion matrix further confirms that the proposed method can effectively predict multiple classes of data under an imbalanced dataset setting, meeting the high-precision requirements for fault diagnosis of avionics equipment.

The proposed method was also experimentally compared with SMOTEBoost, RUSBoost, Adacost, ImECOC, and other methods, including LGBM. Each method was run 10 times to calculate the average value. Table 5 shows the experimental results, and Figure 11 graphically illustrates these outcomes. Additionally, the time cost of the models was compared, using default hyperparameters for each. Table 5 records the corresponding average training and testing times.

The experimental results presented in Table 5 and Figure 11 demonstrate that in the front-end receiver fault diagnosis experiment, the SMOTEWB-LGBM model outperformed other models, achieving a weighted-average precision of 96.05%, a Macro-F1 score of 95.10%, and a Micro-F1 score of 95.92%. These results indicate that the proposed method effectively meets the fault diagnosis requirement of avionics equipment.

Table 6 details the experimental outcomes of the front-end receiver under various data split ratios. It is evident that most algorithms perform optimally with an 8 : 2 data split ratio. This preference is likely due to the advantage of having a larger training set in the context of small-sample data, which provides more information for the model to learn and understand the intrinsic patterns and features of the data. This comprehensive learning typically results in better model generalization during training, leading to enhanced performance on the test set. Furthermore, the SMOTEWB-LGBM method demonstrates consistently high performance across different data split ratios, underscoring its sophistication and robustness.

6. Conclusions

This study introduces the SMOTEWB-LGBM method, effectively addressing the challenge of multiclass imbalance in avionics equipment fault diagnosis. By integrating SMOTEWB oversampling with LGBM's pattern recognition, the method not only improves fault diagnosis accuracy but also enhances the robustness of the system. The SMOTEWB, enhanced with a "successive one-vs-many balancing strategy," optimizes the quality of training samples, thereby enhancing the LGBM model's learning efficiency. Additionally, the use of the TPE method for hyperparameter optimization minimizes manual intervention, further boosting the model's performance. Experimental results demonstrate

superior performance, achieving a Weighted-P of 96.05%, a Macro-F1 score of 95.10%, and a Micro-F1 score of 95.92%, significantly outperforming existing methods like SMOTEBoost, RUSBoost, Adacost, ImECOC, and standalone LGBM.

However, the current application is limited to specific datasets. Future research will expand this method's application to a broader range of avionics equipment and other fault diagnosis areas to verify its generalizability and effectiveness.

Data Availability Statement

The UCI datasets used in this study are openly available in the database at https://archive.ics.uci.edu/datasets (accessed on January 1, 2024).

Conflicts of Interest

The authors declare no conflicts of interest.

Funding

This research was funded by the Mount Taishan Scholar Construction Project in Shandong Province, China, with grant number tstp20221146.

Acknowledgments

We thank the authors of smote-variants, LightGBM, and Optuna open-source code. Our code is based on their open-source code for improvement.

References

[1] Zolghadri, "A review of fault management issues in aircraft systems: current status and future directions," *Progress in Aerospace Sciences*, vol. 147, article 101008, 2024.

[2] S. L. Brunton, K. J. Nathan, and K. Manohar, "Data-driven aerospace engineering: reframing the industry with machine learning," *AIAA Journal*, vol. 59, no. 8, pp. 2820–2847, 2021.

[3] Y. Yuan, J. Wei, H. Huang, W. Jiao, J. Wang, and H. Chen, "Review of resampling techniques for the treatment of imbalanced industrial data classification in equipment condition monitoring," *Engineering Applications of Artificial Intelligence*, vol. 126, article 106911, 2023.

[4] M. S. Santos, P. H. Abreu, N. Japkowicz et al., "On the joint-effect of class imbalance and overlap: a critical review," *Artificial Intelligence Review*, vol. 55, no. 8, pp. 6207–6275, 2022.

[5] Z. Wu, W. Niu, Y. Zhao, and A. Deng, "Key technology and research progress in avionics fault diagnosis," *Proceedings of Fourth International Conference on Machine Learning and Computer Application (ICMLCA 2023)*, vol. 13176, pp. 834–838, 2024.

[6] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.

[7] H. Han, W. Y. Wang, and B. H. Mao, "Borderline-SMOTE: a new over-sampling method in imbalanced data sets learning," in *Proceedings of the 2005 International Conference on Intelligent Computing*, pp. 878–887, Springer Berlin Heidelberg, Berlin, Heidelberg, 2005.

[8] H. B. He, Y. Bai, E. A. Garcia, and S. T. Li, "ADASYN: adaptive synthetic sampling approach for imbalanced learning," in *2008 IEEE International Joint Conference on Neural Networks (IEEE World Congress on Computational Intelligence)*, pp. 1322–1328, Hong Kong, 2008.

[9] F. Sağlam and M. A. Cengiz, "A novel SMOTE-based resampling technique trough noise detection and the boosting procedure," *Expert Systems with Applications*, vol. 200, article 117023, 2022.

[10] Y. Freund and R. E. Schapire, "A decision-theoretic generalization of on-line learning and an application to boosting," *Journal of Computer and System Sciences*, vol. 55, no. 1, pp. 119–139, 1997.

[11] G. Ke, Q. Meng, T. Finley et al., "Lightgbm: a highly efficient gradient boosting decision tree," *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[12] Q. Li, Y. Wang, Y. Shao, L. Li, and H. Hao, "A comparative study on the most effective machine learning model for blast loading prediction: from GBDT to transformer," *Engineering Structures*, vol. 276, article 115310, 2023.

[13] B. Xi, E. Li, Y. Fissha, J. Zhou, and P. Segarra, "LGBM-based modeling scenarios to compressive strength of recycled aggregate concrete with SHAP analysis," *Mechanics of Advanced Materials and Structures*, vol. 30, pp. 1–16, 2023.

[14] S. Prusty, S. Patnaik, and S. K. Dash, "SKCV: stratified K-fold cross-validation on ML classifiers for predicting cervical cancer," *Frontiers in Nanotechnology*, vol. 4, article 972421, 2022.

[15] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kegl, "Algorithms for hyper-parameter optimization," *Advances in Neural Information Processing Systems*, vol. 24, 2011.

[16] G. Soman, M. V. Vivek, M. V. Judy, E. Papageorgiou, and V. C. Gerogiannis, "Precision-based weighted blending distributed ensemble model for emotion classification," *Algorithms*, vol. 15, no. 2, p. 55, 2022.

[17] Z. Feng, K. Mao, and H. Zhou, "Adaptive micro- and macro-knowledge incorporation for hierarchical text classification," *Expert Systems with Applications*, vol. 248, article 123374, 2024.

[18] N. V. Chawla, A. Lazarevic, L. O. Hall, and K. W. Bowyer, "SMOTEBoost: improving prediction of the minority class in boosting," in *Proceedings of 7th European Conference on Principles and Practice of Knowledge Discovery in Databases(Knowledge Discovery in Databases: PKDD 2003)*, pp. 107–119, Cavtat-Dubrovnik, Croatia, 2003.

[19] T. Seiffert, M. Khoshgoftaar, J. V. Hulse, and A. Napolitano, "RUSBoost: improving classification performance when training data is skewed," in *2008 19th International Conference on Pattern Recognition*, Tampa, FL, USA, 2008.

[20] J. Zhang, "AdaCost: misclassification cost-sensitive boosting," *Proceedings of the Sixteenth International Conference on Machine Learning (ICML 1999)*, vol. 99, pp. 97–105, 1999.

[21] X. Y. Liu, Q. Q. Li, and Z. H. Zhou, "Learning imbalanced multi-class data with optimal dichotomy weights," in *Proceedings of 2013 IEEE 13th International Conference on Data Mining*, pp. 478–487, Dallas, USA, 2013.

[22] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: a next-generation hyperparameter optimization framework," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD '19)*, pp. 2623–2631, Anchorage, AK, USA, 2019.